PROBLEM STATEMENT :

The Reserve Bank of India (RBI) aims to monitor the performance of the top 5 banks in India to optimize resource allocation, enhance customer services, and identify opportunities for new financial products or schemes that can increase revenue. Currently, the evaluation of bank performance and loan allocation is largely manual and reactive, leading to inefficiencies and missed growth opportunities so predict the optimal loan amount for individual customers based on their financial profile, reducing risk and maximizing profitability.

1. Importing Libraries & Data :

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [13]:
```
df = pd.read_excel("India Bank.xlsx", sheet_name='LoanPrediction')
df
```

Out[13]:

	Customer Name	Age	Bank Name	Fixed Deposit	Properties Owned	Salary	Tendency	Savings Account	Bank Loan
0	Aarav Patil	61	SBI	482044	1	104195	Spender	117740.35	323004.50
1	Tanvi Deshmukh	50	Axis Bank	389668	3	21279	Spender	29152.23	69582.33
2	Siddharth Shinde	33	SBI	426355	3	24125	Spender	13027.50	48250.00
3	Riya Jadhav	34	SBI	372307	6	57833	Spender	65929.62	131859.24
4	Aditya Pawar	60	SBI	414732	1	75753	Spender	42421.68	278013.51
...	...	...	...	...	...	...	...	...	...
10137	Varsha Ghodgade	63	HDFC Bank	356271	4	57836	Neutral	81548.76	86754.00
10138	Varun Pisalkar	56	Axis Bank	387972	1	15919	Saver	28017.44	42503.73
10139	Vasudha Dagade	40	ICICI Bank	358570	3	114964	Neutral	170146.72	344892.00
10140	Vedant Tupkar	55	ICICI Bank	449849	1	41063	Spender	16014.57	134276.01
10141	Vedika Shintre	36	SBI	362351	5	85629	Saver	145569.30	320252.46

10142 rows × 9 columns

2. EDA Process :

In [14]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10142 entries, 0 to 10141
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Customer Name     10142 non-null  object
 1   Age               10142 non-null  int64
 2   Bank Name         10142 non-null  object
 3   Fixed Deposit     10142 non-null  int64
 4   Properties Owned  10142 non-null  int64
 5   Salary            10142 non-null  int64
 6   Tendency          10142 non-null  object
 7   Savings Account   10142 non-null  float64
 8   Bank Loan         10142 non-null  float64
dtypes: float64(2), int64(4), object(3)
memory usage: 713.2+ KB
```

In [15]:
```
df.duplicated()
```

Out[15]:
```
0        False
1        False
2        False
3        False
4        False
         ...
10137    False
10138    False
10139    False
10140    False
10141    False
Length: 10142, dtype: bool
```

In [16]:
```
df = df.drop(columns=['Customer Name', 'Tendency'], axis=1)
```

In [17]:
```
df
```

Out[17]:

	Age	Bank Name	Fixed Deposit	Properties Owned	Salary	Savings Account	Bank Loan
0	61	SBI	482044	1	104195	117740.35	323004.50
1	50	Axis Bank	389668	3	21279	29152.23	69582.33
2	33	SBI	426355	3	24125	13027.50	48250.00
3	34	SBI	372307	6	57833	65929.62	131859.24
4	60	SBI	414732	1	75753	42421.68	278013.51
...	...	...	...	...	...	...	...
10137	63	HDFC Bank	356271	4	57836	81548.76	86754.00
10138	56	Axis Bank	387972	1	15919	28017.44	42503.73
10139	40	ICICI Bank	358570	3	114964	170146.72	344892.00
10140	55	ICICI Bank	449849	1	41063	16014.57	134276.01
10141	36	SBI	362351	5	85629	145569.30	320252.46

10142 rows × 7 columns

In [18]:
```
df['Savings Account'] = df['Savings Account'].astype('int64')
```

In [19]:
```
df['Bank Loan'] = df['Bank Loan'].astype('int64')
```

In [20]:
```
df
```

Out[20]:

	Age	Bank Name	Fixed Deposit	Properties Owned	Salary	Savings Account	Bank Loan
0	61	SBI	482044	1	104195	117740	323004
1	50	Axis Bank	389668	3	21279	29152	69582
2	33	SBI	426355	3	24125	13027	48250
3	34	SBI	372307	6	57833	65929	131859
4	60	SBI	414732	1	75753	42421	278013
...	...	...	...	...	...	...	...
10137	63	HDFC Bank	356271	4	57836	81548	86754
10138	56	Axis Bank	387972	1	15919	28017	42503
10139	40	ICICI Bank	358570	3	114964	170146	344892
10140	55	ICICI Bank	449849	1	41063	16014	134276
10141	36	SBI	362351	5	85629	145569	320252

10142 rows × 7 columns

In [21]:
```
df = pd.get_dummies(df, columns=['Bank Name'], dtype='int64')
```

In [22]:
```
df
```

Out[22]:

	Age	Fixed Deposit	Properties Owned	Salary	Savings Account	Bank Loan	Bank Name_Axis Bank	Bank Name_HDFC Bank	Bank Name_ICICI Bank	Bank Name_Kotak Mahindra Bank	Bank Name_SBI
0	61	482044	1	104195	117740	323004	0	0	0	0	1
1	50	389668	3	21279	29152	69582	1	0	0	0	0
2	33	426355	3	24125	13027	48250	0	0	0	0	1
3	34	372307	6	57833	65929	131859	0	0	0	0	1
4	60	414732	1	75753	42421	278013	0	0	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...
10137	63	356271	4	57836	81548	86754	0	1	0	0	0
10138	56	387972	1	15919	28017	42503	1	0	0	0	0
10139	40	358570	3	114964	170146	344892	0	0	1	0	0
10140	55	449849	1	41063	16014	134276	0	0	1	0	0
10141	36	362351	5	85629	145569	320252	0	0	0	0	1

10142 rows × 11 columns

----------Log Transformation----------

In [23]:
```
df['Bank Loan'].skew()
```

Out[23]:
```
np.float64(0.675475960681291)
```

In [24]:
```
df['BankLoan_log'] = np.log1p(df['Bank Loan'])
```

In [25]:
```
df.head()
```

Out[25]:

	Age	Fixed Deposit	Properties Owned	Salary	Savings Account	Bank Loan	Bank Name_Axis Bank	Bank Name_HDFC Bank	Bank Name_ICICI Bank	Bank Name_Kotak Mahindra Bank	Bank Name_SBI	BankLoan_log
0	61	482044	1	104195	117740	323004	0	0	0	0	1	12.685423
1	50	389668	3	21279	29152	69582	1	0	0	0	0	11.150276
2	33	426355	3	24125	13027	48250	0	0	0	0	1	10.784172
3	34	372307	6	57833	65929	131859	0	0	0	0	1	11.789496
4	60	414732	1	75753	42421	278013	0	0	0	0	1	12.535427

3. Spliting the data into training & testing part :

In [26]:
```
x = df.drop(['Bank Loan', 'BankLoan_log'], axis=1)
y = df['BankLoan_log']
```

In [27]:
```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.2, random_state=42)
```

4. Fitting the data into model :

------Tuned the parameters of RandomForestRegressor to optimize its performance------

In [28]:
```
from sklearn.ensemble import RandomForestRegressor
rfr = RandomForestRegressor()

rfr = RandomForestRegressor(n_estimators=300, max_depth=10, min_samples_split=5, min_samples_leaf=2, random_state=42, n_jobs=-1)

rfr.fit(x_train, y_train)
```

Out[28]:
```
RandomForestRegressor
RandomForestRegressor(max_depth=10, min_samples_leaf=2, min_samples_split=5,
                      n_estimators=300, n_jobs=-1, random_state=42)
```

5. Model Evaluation :

In [29]:
```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

train_pred = rfr.predict(x_train)
test_pred  = rfr.predict(x_test)

print("Train R2:", r2_score(y_train, train_pred))
print("Test R2 :", r2_score(y_test, test_pred))
print("MAE     :", mean_absolute_error(y_test, test_pred))
print("RMSE    :", np.sqrt(mean_squared_error(y_test, test_pred)))
```

```
Train R2: 0.8031142242864363
Test R2 : 0.706069961989368
MAE     : 0.3290599222960491
RMSE    : 0.3873468842198965
```

The model achieved an R^2 of 0.80 on training data and 0.70 on test data, indicating good generalization with minimal overfitting. The MAE of 0.33 and RMSE of 0.38 suggest that prediction errors are relatively small and consistent, with no significant extreme deviations.

6. Converting Values to original scale :

In [30]:
```
y_pred_log = rfr.predict(x_test)
y_pred = np.expm1(y_pred_log)
y_pred
```

Out[30]:
```
array([348090.26832299, 118325.95071004,  56492.64228272, ...,
       220070.0235888 , 250117.46827181, 183451.53450102])
```

7. Cross Validation :

In [31]:
```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(rfr, x, y, cv=5, scoring='r2')

print("CV R2 Mean:", cv_scores.mean())
```

```
CV R2 Mean: 0.698874138342908
```

8. Saving the trained model :

In [ ]:
```
import joblib

joblib.dump(rfr, "rf_loan_model.pkl")
```

Out[ ]:
```
['rf_loan_model.pkl']
```